

Fast Dense Solvers for Spectral Physics-Informed Neural Networks via Gauss–Newton Compression

A Sherman–Morrison–Woodbury and Matrix-Free LSQR Study
on One-Dimensional Boundary-Value Problems

Research Report

May 16, 2026

Abstract

We study Gauss–Newton (GN) optimisation of spectral physics-informed neural networks (PINNs) for two boundary-value problems on $[-1, 1]$ with homogeneous Dirichlet conditions: the linear problem $-u'' = f$ and the nonlinear problem $-u'' + u^3 = f$. The solution is parameterised as $u_\theta(x) = \Phi(x) c(\theta)$ where Φ is a Chebyshev basis evaluated at Chebyshev–Gauss–Lobatto (CGL) collocation nodes and $c(\theta) \in \mathbb{R}^K$ is the output of a small multilayer perceptron with P parameters. Because $\text{rank}(\partial r / \partial \theta) \leq K \ll P$, the GN normal equations are rank-deficient and a Levenberg–Marquardt (LM) damping is required. We compare three solvers of the damped normal system: (i) a strong dense baseline using LAPACK `lstsq` (linear) or SciPy `least_squares` with the analytic Jacobian (nonlinear); (ii) a Sherman–Morrison–Woodbury (SMW) compression that reduces the $P \times P$ system to a $K \times K$ solve; and (iii) a matrix-free LSQR solver that never materialises the Jacobian. On the nonlinear problem at $N = 2000$ collocation points, the SMW method is $209\times$ **faster** than the dense baseline (786ms vs 164s) at comparable relative L^2 accuracy ($\sim 10^{-6}$ vs 10^{-11}). The matrix-free LSQR variant gives a $111\times$ speedup at the same N and is the structural choice when K is large. This report documents the full methodology, gives the derivation of both compressed solvers, lists the complete implementation, and analyses the speed–accuracy trade-offs.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Contributions	3
1.3	Notation	3
2	Problem Setup	3
2.1	The two BVPs	3
2.2	Chebyshev pseudospectral discretisation	3
2.3	Why a small Chebyshev basis suffices	4
2.4	Spectral PINN parameterisation	4
3	Gauss–Newton Normal Equations and Their Structure	4
3.1	The key rank observation	5
3.2	Cheap incremental updates to G_K in the nonlinear case	5
4	Three Solvers for the Damped Normal Equations	5
4.1	Method 0 (M0): Dense baseline	5
4.2	Method 1 (M1): Sherman–Morrison–Woodbury compression	5
4.3	Method 2 (M2): Matrix-free LSQR	6
4.4	Shared Levenberg–Marquardt outer loop	7

5	Implementation	7
5.1	Chebyshev pseudospectral machinery	7
5.2	Problem definitions and the analytic Jacobian	8
5.3	Coefficient network and vectorised $B = \partial c / \partial \theta$	8
5.4	The three solvers	9
5.5	Shared LM outer loop	11
6	Experimental Results	12
6.1	Experimental setup	12
6.2	Adjoint consistency	13
6.3	Headline benchmark results	13
6.4	Sanity-check runs at $N = 80$	16
7	Analysis and Discussion	16
7.1	Complexity, in detail	16
7.2	Why M1 wins over M2 at $K = 20$	16
7.3	Accuracy and the conditioning of G_K	17
7.4	The dense baseline’s failure at $N = 2000$ linear	17
7.5	Levenberg–Marquardt convergence pattern	17
7.6	Scope and caveats	17
8	Reproducibility	18
9	Conclusion	18

1 Introduction

1.1 Motivation

Physics-informed neural networks parameterise the solution of a PDE as a neural network and minimise the PDE residual on a collocation grid. The natural loss is a least-squares functional, which motivates second-order optimisation: Gauss–Newton or Levenberg–Marquardt. However, when the network has P parameters and the collocation grid has N points, the GN normal equations involve $P \times P$ matrices that are prohibitively expensive to assemble and factor at every outer iteration.

A specific structure rescues us in the spectral PINN setting. If the network output is the coefficient vector of a fixed truncated polynomial expansion

$$u_\theta(x) = \Phi(x) c(\theta), \quad c(\theta) \in \mathbb{R}^K, \quad (1)$$

with K fixed and much smaller than P , then the Jacobian of the residual with respect to θ factors through a K -dimensional bottleneck:

$$J = J_u(u) \Phi B, \quad B := \partial c / \partial \theta \in \mathbb{R}^{K \times P}. \quad (2)$$

Consequently $\text{rank}(J) \leq K \ll P$, and the damped normal equations $(B^\top G_K B + \mu I_P) \Delta \theta = -B^\top h_K$ admit *Woodbury-type* compression to a $K \times K$ solve.

1.2 Contributions

1. We give a clean derivation of the SMW-compressed Gauss–Newton step for spectral PINNs (Section 4.2), in a form that avoids forming G_K^{-1} and avoids any inner line search by relying on adaptive LM damping. We exploit the structure $J_u = A_{\text{lin}} + E$ where E is a u -dependent diagonal supported on the PDE rows to reduce G_K updates to $\mathcal{O}((N+1)K)$ per iteration.
2. We give the corresponding matrix-free LSQR formulation (Section 4.3) with explicit forward and adjoint operators of cost $\mathcal{O}(KP + (N+1)K)$ each, and verify adjoint consistency to round-off.
3. We benchmark all three solvers on the linear and nonlinear BVPs across nine grid sizes $N \in \{40, 80, 120, 160, 250, 500, 1000, 1500, 2000\}$. The empirical scaling matches the predicted $\mathcal{O}(N)$ vs $\mathcal{O}(N^3)$ cost, giving $209\times$ and $111\times$ speedup at $N = 2000$ for the SMW and LSQR methods respectively.
4. We document the failure mode of the dense baseline at $N = 2000$ in the linear case (LAPACK `gelsd` returns garbage) and explain why the network reparameterisation is itself a regularisation.

1.3 Notation

Lowercase bold-italic letters denote vectors and uppercase Roman letters denote matrices. $\Phi \in \mathbb{R}^{(N+1) \times K}$ is the Chebyshev basis matrix evaluated at the CGL nodes; $A_{\text{lin}} \in \mathbb{R}^{(N+3) \times (N+1)}$ is the discrete linear operator augmented with two boundary-condition rows of weight λ . We use $\text{diag}_{\text{ext}}(v)$ for the embedding of $v \in \mathbb{R}^{N+1}$ as a diagonal supported on the top $N+1$ rows of an $(N+3) \times (N+1)$ block. The Frobenius norm is $\|\cdot\|_F$ and the spectral ℓ^2 norm is $\|\cdot\|_2$.

2 Problem Setup

2.1 The two BVPs

Both problems are posed on $x \in [-1, 1]$ with homogeneous Dirichlet boundary conditions $u(\pm 1) = 0$. We use the manufactured solution $u^*(x) = \sin(\pi x)$ so that exact-solution errors can be measured.

Problem A (linear).

$$-u''(x) = f(x), \quad f(x) = \pi^2 \sin(\pi x). \quad (3)$$

Problem B (nonlinear).

$$-u''(x) + u(x)^3 = f(x), \quad f(x) = \pi^2 \sin(\pi x) + \sin^3(\pi x). \quad (4)$$

2.2 Chebyshev pseudospectral discretisation

We collocate on the $N+1$ Chebyshev–Gauss–Lobatto (CGL) nodes,

$$x_j = \cos\left(\frac{j\pi}{N}\right), \quad j = 0, 1, \dots, N, \quad (5)$$

and use the standard CGL differentiation matrix $D \in \mathbb{R}^{(N+1) \times (N+1)}$ [1, Eq. 6.4]. The PDE operator is $-D^2$, and we encode the two boundary conditions by augmenting with weighted

identity rows:

$$A_{\text{lin}} = \begin{bmatrix} -D^2 \\ \lambda e_0^\top \\ \lambda e_N^\top \end{bmatrix} \in \mathbb{R}^{(N+3) \times (N+1)}, \quad \lambda = 10. \quad (6)$$

The discrete RHS is $\tilde{f} = [f(x_0), \dots, f(x_N), 0, 0]^\top \in \mathbb{R}^{N+3}$. The nonlinear discrete residual is

$$F(u) = A_{\text{lin}} u + g(u) - \tilde{f}, \quad g(u) = \begin{cases} [u^3; 0; 0] \in \mathbb{R}^{N+3} & \text{(nonlinear)} \\ 0 & \text{(linear)}. \end{cases} \quad (7)$$

The Jacobian of F with respect to u is

$$J_u(u) = A_{\text{lin}} + E(u), \quad E(u) = \text{diag}_{\text{ext}}(3u^2). \quad (8)$$

Here $\text{diag}_{\text{ext}}(v)$ denotes the $(N+3) \times (N+1)$ matrix whose top $N+1 \times N+1$ block is $\text{diag}(v)$ and whose bottom two rows are zero; $E \equiv 0$ in the linear case.

2.3 Why a small Chebyshev basis suffices

For analytic u^* , the Chebyshev expansion converges geometrically. For $u^*(x) = \sin(\pi x)$ in particular, machine precision is reached already at $K \approx 15$. We verify this empirically by projecting u^* onto Φ via least squares and reporting $\|\Phi c - u^*\|_2 / \|u^*\|_2$:

K	projection error
4	1.7×10^{-2}
8	4.6×10^{-7}
12	1.6×10^{-12}
16	2.5×10^{-16}
20	2.5×10^{-16}
30	4.2×10^{-16}

This is the structural fact that makes the SMW compression effective: **we can fix $K = 20$ for every collocation grid size N** , so the $K \times K$ solve in the SMW method has bounded cost while N grows.

2.4 Spectral PINN parameterisation

The numerical solution is $u_\theta = \Phi c(\theta) \in \mathbb{R}^{N+1}$, where $c : \mathbb{R}^P \rightarrow \mathbb{R}^K$ is a small multilayer perceptron fed a constant input,

$$c(\theta) = W_3 \tanh(W_2 \tanh(W_1 \cdot 1 + b_1) + b_2) + b_3, \quad (9)$$

with hidden width $H = 8$ and $P \approx 200$ parameters. We zero-initialise the output layer ($W_3 = 0$, $b_3 = 0$) so that the initial coefficient vector is exactly zero and the first GN step starts from $u \equiv 0$. This is practically important at large N where the residual norm $\|\tilde{f}\|_2$ is large and a random initialisation can give an initial residual that overflows the LM trust region.

The Jacobian $B := \partial c / \partial \theta \in \mathbb{R}^{K \times P}$ is computed in one vectorised reverse-mode call with `torch.func.jacrev`, replacing the naive K -step backward loop.

3 Gauss–Newton Normal Equations and Their Structure

The residual we minimise is $r(\theta) := F(\Phi c(\theta)) \in \mathbb{R}^{N+3}$. By the chain rule,

$$J(\theta) = \frac{\partial r}{\partial \theta} = J_u(\Phi c(\theta)) \Phi B(\theta) \in \mathbb{R}^{(N+3) \times P}. \quad (10)$$

3.1 The key rank observation

Proposition 1 (Low-rank factorisation). *For every θ , $\text{rank}(J(\theta)) \leq K$.*

Proof. $J = (J_u \Phi) B$ where $J_u \Phi \in \mathbb{R}^{(N+3) \times K}$ and $B \in \mathbb{R}^{K \times P}$. The rank of a product is bounded by the smaller of the inner dimensions. $\square$

Since $K \ll P$, the Gauss–Newton normal equations $J^\top J \Delta\theta = -J^\top r$ form a $P \times P$ system of rank at most K . Direct factorisation of this $P \times P$ system is wasteful; the Levenberg–Marquardt damping reads

$$(B^\top G_K B + \mu I_P) \Delta\theta = -B^\top h_K, \quad (11)$$

where we have introduced the K -dimensional Gram matrix and gradient

$$G_K := (J_u \Phi)^\top (J_u \Phi) \in \mathbb{R}^{K \times K}, \quad h_K := \Phi^\top J_u^\top r \in \mathbb{R}^K. \quad (12)$$

3.2 Cheap incremental updates to G_K in the nonlinear case

In the linear case $J_u = A_{\text{lin}}$ is constant, so $G_K = (A_{\text{lin}} \Phi)^\top (A_{\text{lin}} \Phi) =: G_{\text{lin}}$ is computed once.

In the nonlinear case, expand $J_u \Phi = A_{\text{lin}} \Phi + E \Phi$ to obtain

$$G_K = G_{\text{lin}} + \Phi^\top A_{\text{lin}}^\top E \Phi + (\Phi^\top A_{\text{lin}}^\top E \Phi)^\top + \Phi^\top E^\top E \Phi. \quad (13)$$

Because E is diagonal with support only on the top $N+1$ rows, each cross-term reduces to operations restricted to the top $N+1 \times N+1$ block of A_{lin} . The dominant work is one $(N+1) \times K$ matrix multiplication per cross-term, costing $\mathcal{O}((N+1)K^2)$ in total. The right-hand side h_K admits the analogous decomposition

$$h_K = \Phi^\top A_{\text{lin}}^\top r + \Phi^\top (e \odot r_{1:N+1}), \quad e = 3u^2. \quad (14)$$

4 Three Solvers for the Damped Normal Equations

4.1 Method 0 (M0): Dense baseline

For Problem A we solve $A_{\text{lin}} u = \tilde{f}$ directly with `numpy.linalg.lstsq` (LAPACK `gelsd`). For Problem B we run `scipy.optimize.least_squares` with method `'lm'` and the *analytic* Jacobian J_u from (8); this is a strong baseline because it avoids finite-difference Jacobian evaluations. Cost per LM step is $\mathcal{O}(N^3)$ from the dense factorisation of the $(N+3) \times (N+1)$ Jacobian.

4.2 Method 1 (M1): Sherman–Morrison–Woodbury compression

Lemma 1 (Woodbury compression). *Let $G_K \in \mathbb{R}^{K \times K}$ be symmetric positive semidefinite, $B \in \mathbb{R}^{K \times P}$, $g_P \in \mathbb{R}^P$, and $\mu > 0$. Define*

$$M := \mu G_K^{-1} + B B^\top \in \mathbb{R}^{K \times K}. \quad (15)$$

Then the unique solution $\Delta\theta \in \mathbb{R}^P$ of $(\mu I_P + B^\top G_K B) \Delta\theta = -g_P$ satisfies

$$\Delta\theta = -\frac{1}{\mu} (g_P - B^\top y), \quad M y = B g_P. \quad (16)$$

Furthermore, the system $M y = B g_P$ can be rewritten without forming G_K^{-1} as

$$(\mu I_K + G_K B B^\top) y = G_K B g_P. \quad (17)$$

Proof. The Sherman–Morrison–Woodbury identity gives $(\mu I_P + B^\top G_K B)^{-1} = \frac{1}{\mu} [I_P - B^\top M^{-1} B]$ with M as in (15). Therefore

$$\begin{aligned}\Delta\theta &= -\frac{1}{\mu} [I - B^\top M^{-1} B] g_P \\ &= -\frac{1}{\mu} [g_P - B^\top (M^{-1} B g_P)] \\ &= -\frac{1}{\mu} [g_P - B^\top y],\end{aligned}$$

where $y = M^{-1} B g_P$ solves $My = B g_P$. Left-multiplying M in this last equation by G_K gives (17). $\square$

Remark 1. Equation (17) is the workhorse. We never compute G_K^{-1} , and the matrix $\mu I_K + G_K B B^\top$ is small ($K \times K$) and well-conditioned for moderate μ . With $g_P = B^\top h_K$, the M1 step is summarised:

1. Form $g_P = B^\top h_K$, $B g_P \in \mathbb{R}^K$, and $B B^\top \in \mathbb{R}^{K \times K}$.
2. Build G_K via the structural update (13).
3. Solve $(\mu I_K + G_K B B^\top) y = G_K B g_P$.
4. Return $\Delta\theta = -\frac{1}{\mu} (g_P - B^\top y)$.

Total cost per LM step is $\mathcal{O}(K^3 + K^2 P + (N+1)K^2)$, which for fixed K and growing N is $\mathcal{O}(N)$.

4.3 Method 2 (M2): Matrix-free LSQR

LSQR (Paige–Saunders, 1982) solves the damped least-squares problem

$$\min_{\Delta\theta} \left\| \begin{bmatrix} J \\ \sqrt{\mu} I_P \end{bmatrix} \Delta\theta - \begin{bmatrix} -r \\ 0 \end{bmatrix} \right\|_2^2 \quad (18)$$

using only matvecs with J and $J^\top$. Define the augmented forward operator $Q : \mathbb{R}^P \rightarrow \mathbb{R}^{N+3+P}$,

$$Qx = \begin{bmatrix} A_{\text{lin}} \Phi(Bx) + E \Phi(Bx) \\ \sqrt{\mu} x \end{bmatrix}. \quad (19)$$

A direct calculation gives the adjoint

$$Q^\top y = B^\top \left[(A_{\text{lin}} \Phi)^\top y_1 + \Phi^\top (E y_{1,\text{top}}) \right] + \sqrt{\mu} y_2, \quad (20)$$

where $y = (y_1; y_2)$ with $y_1 \in \mathbb{R}^{N+3}$, $y_2 \in \mathbb{R}^P$, and $y_{1,\text{top}}$ denotes the first $N+1$ entries of y_1 .

Proposition 2 (Adjoint correctness). *For all $x \in \mathbb{R}^P$ and $y \in \mathbb{R}^{N+3+P}$, $\langle Qx, y \rangle = \langle x, Q^\top y \rangle$.*

Proof. Expand both sides using $\langle \cdot, \cdot \rangle$ on the appropriate Euclidean spaces:

$$\begin{aligned}\langle Qx, y \rangle &= \langle A_{\text{lin}} \Phi Bx + E \Phi Bx, y_1 \rangle + \sqrt{\mu} \langle x, y_2 \rangle \\ &= \langle Bx, (A_{\text{lin}} \Phi)^\top y_1 \rangle + \langle \Phi Bx, E y_1 \rangle + \sqrt{\mu} \langle x, y_2 \rangle \\ &= \langle x, B^\top (A_{\text{lin}} \Phi)^\top y_1 \rangle + \langle x, B^\top \Phi^\top (E y_{1,\text{top}}) \rangle + \sqrt{\mu} \langle x, y_2 \rangle \\ &= \langle x, Q^\top y \rangle,\end{aligned}$$

using $E y_1$ has support on the top $N+1$ entries only. $\square$

We verify the identity numerically with random Gaussian x, y — the test passes to $\sim 10^{-11}$ for both problems at $N = 40$ (Section 6). Costs per matvec:

$$\text{forward: } \mathcal{O}(KP + (N+1)K), \quad \text{adjoint: } \mathcal{O}(KP + (N+1)K).$$

With $K_{\text{LSQR}} = K + 15 = 35$ inner LSQR iterations (chosen to fully resolve the K -dimensional Krylov subspace), one M2 LM step costs $\mathcal{O}(K_{\text{LSQR}}(KP + (N+1)K))$, again $\mathcal{O}(N)$ for fixed K and bounded P .

4.4 Shared Levenberg–Marquardt outer loop

Both M1 and M2 share an adaptive LM outer loop:

1. Initialise θ_0 with the zero-output-layer scheme; compute r_0, u_0, B_0 .
2. For $k = 0, 1, 2, \dots$:
 - Build $\Delta\theta_k$ from the current μ_k via SMW or LSQR.
 - If $\|\Delta\theta_k\|_2 > \tau_{\text{step}}$, rescale.
 - Compute trial $\theta_k + \Delta\theta_k$ and trial residual.
 - **If residual decreases:** accept, $\mu_{k+1} = \rho_{\downarrow}\mu_k$, refresh B .
 - **Otherwise:** reject, $\mu_{k+1} = \rho_{\uparrow}\mu_k$, retry.
3. Stop when $\|r_k\|_2 < \tau_{\text{res}}$ or the relative decrease drops below τ_{rel} .

Hyperparameters (validated in Section 6): $\mu_0 = 1.0$, $\rho_{\uparrow} = 3.0$, $\rho_{\downarrow} = 0.5$, $\tau_{\text{res}} = 10^{-9}$, $\tau_{\text{rel}} = 10^{-14}$, $\tau_{\text{step}} = 10^3$, max LM steps = 80.

No inner line search is required: the adaptive μ plays the role of a trust region. The accept/reject rule is the only mechanism needed for global convergence on these problems.

5 Implementation

The complete implementation is in a single Python module `spectral_pinn.py`. All computation is performed in `float64`.

5.1 Chebyshev pseudospectral machinery

```
1 def chebyshev_cgl(N: int) -> torch.Tensor:
2     """Return the N+1 CGL nodes on [-1, 1]."""
3     j = torch.arange(N + 1, dtype=torch.float64)
4     return torch.cos(torch.pi * j / N)
5
6
7 def chebyshev_diff_matrix(N: int):
8     """Return (x, D) where D is the (N+1)x(N+1) CGL differentiation matrix."""
9     x = chebyshev_cgl(N)
10    c = torch.ones(N + 1, dtype=torch.float64)
11    c[0] = c[-1] = 2.0
12
13    X = x.unsqueeze(0)
14    dX = X.T - X + torch.eye(N + 1)
15
16    i_idx = torch.arange(N + 1).unsqueeze(1)
17    j_idx = torch.arange(N + 1).unsqueeze(0)
18    sign = (-1.0) ** (i_idx + j_idx)
19    C_ratio = c.unsqueeze(1) / c.unsqueeze(0)
20
21    D = C_ratio * sign / dX
22    D = D - torch.diag(D.sum(dim=1))
23    return x, D
24
25
26 def chebyshev_basis(x: torch.Tensor, K: int) -> torch.Tensor:
27     """Evaluate T_0, ..., T_{K-1} at x. Shape (len(x), K)."""
28     theta = torch.arccos(x.clamp(-1 + 1e-14, 1 - 1e-14))
29     k = torch.arange(K, dtype=torch.float64)
```

```

30     return torch.cos(theta.unsqueeze(1) * k.unsqueeze(0))
31
32
33 def build_A_lin(D: torch.Tensor, bc_weight: float = 10.0) -> torch.Tensor:
34     """A_lin = [ -D^2 ; lambda e_0^T ; lambda e_N^T ]   shape (N+3, N+1)."""
35     N = D.shape[0] - 1
36     A_pde = -(D @ D)
37     bc = torch.zeros(2, N + 1, dtype=torch.float64)
38     bc[0, 0] = bc_weight
39     bc[1, -1] = bc_weight
40     return torch.cat([A_pde, bc], dim=0)

```

Listing 1: CGL nodes, differentiation matrix, basis, and A_{lin} .

5.2 Problem definitions and the analytic Jacobian

```

1 def true_solution(x):     return torch.sin(torch.pi * x)
2 def rhs_linear(x):        return (torch.pi ** 2) * torch.sin(torch.pi * x)
3 def rhs_nonlinear(x):
4     s = torch.sin(torch.pi * x)
5     return (torch.pi ** 2) * s + s ** 3
6
7
8 def build_rhs(x: torch.Tensor, problem: str) -> torch.Tensor:
9     f = rhs_linear(x) if problem == "linear" else rhs_nonlinear(x)
10    return torch.cat([f, torch.zeros(2, dtype=torch.float64)])
11
12
13 def residual_F(u, A_lin, f_tilde, problem):
14     Fu = A_lin @ u
15     if problem == "nonlinear":
16         Np1 = u.shape[0]
17         nu = torch.zeros_like(Fu)
18         nu[:Np1] = u ** 3
19         Fu = Fu + nu
20     return Fu - f_tilde
21
22
23 def linearised_operator(u, A_lin, problem):
24     """J_u = A_lin + diag_ext(3 u^2)."""
25     Ju = A_lin.clone()
26     if problem == "nonlinear":
27         Np1 = u.shape[0]
28         idx = torch.arange(Np1)
29         Ju[idx, idx] = Ju[idx, idx] + 3.0 * u ** 2
30     return Ju

```

Listing 2: Residual $F(u)$ and linearised operator J_u .

5.3 Coefficient network and vectorised $B = \partial c / \partial \theta$

```

1 class CoeffNet(nn.Module):
2     def __init__(self, K, hidden=8, zero_init_output=True):
3         super().__init__()
4         self.K = K
5         self.hidden = hidden
6         self.net = nn.Sequential(
7             nn.Linear(1, hidden), nn.Tanh(),
8             nn.Linear(hidden, hidden), nn.Tanh(),
9             nn.Linear(hidden, K),

```



```

10         )
11         linear_layers = [m for m in self.modules() if isinstance(m, nn.Linear)]
12         for m in linear_layers[:-1]:
13             nn.init.xavier_uniform_(m.weight)
14             nn.init.zeros_(m.bias)
15         last = linear_layers[-1]
16         if zero_init_output:
17             nn.init.zeros_(last.weight)
18             nn.init.zeros_(last.bias)
19         else:
20             nn.init.xavier_uniform_(last.weight)
21             nn.init.zeros_(last.bias)
22
23     def forward(self):
24         return self.net(torch.ones(1, 1, dtype=torch.float64)).view(-1)
25
26
27 def build_jacobian_B(model):
28     """B = dc/dtheta in R^{K x P}, computed in one vectorised pass."""
29     params = dict(model.named_parameters())
30
31     def fwd(p):
32         return torch.func.functional_call(model, p, {}, {})
33
34     jac_dict = torch.func.jacrev(fwd)(params)
35     K = model.K
36     blocks = [jac_dict[name].reshape(K, -1)
37               for name, _ in model.named_parameters()]
38     return torch.cat(blocks, dim=1)

```

Listing 3: MLP for $c(\theta)$ and vectorised Jacobian via `torch.func.jacrev`.

5.4 The three solvers

```

1 def run_method_0(N, problem, bc_weight=10.0, lm_max_nfev=200):
2     t0 = time.perf_counter()
3     x, D = chebyshev_diff_matrix(N)
4     A_lin = build_A_lin(D, bc_weight=bc_weight)
5     f_tilde = build_rhs(x, problem)
6     A_np, f_np = A_lin.numpy(), f_tilde.numpy()
7     t1 = time.perf_counter()
8
9     if problem == "linear":
10         u_np, *_ = np.linalg.lstsq(A_np, f_np, rcond=None)
11     else:
12         def fun(u_np):
13             return residual_F(torch.from_numpy(u_np), A_lin,
14                               f_tilde, problem).numpy()
15         def jac(u_np):
16             return linearised_operator(torch.from_numpy(u_np),
17                                       A_lin, problem).numpy()
18         sol = least_squares(fun, np.zeros(N + 1), jac=jac, method="lm",
19                             max_nfev=lm_max_nfev,
20                             xtol=1e-12, ftol=1e-12, gtol=1e-12)
21         u_np = sol.x
22         t2 = time.perf_counter()
23         # ... compute L2 error, return result dict ...

```

Listing 4: Method 0: dense baseline (linear via `lstsq`, nonlinear via SciPy LM with analytic Jacobian).

```

1 def gn_normal_pieces(u, r, A_lin, Phi, G_lin, problem):
2     """Build (G_K, h_K) by exploiting J_u = A_lin + E."""
3     if problem == "linear":
4         h_K = Phi.T @ (A_lin.T @ r)
5         return G_lin, h_K
6
7     Np1 = u.shape[0]
8     e = 3.0 * u ** 2                                # diag of E (top N+1 entries)
9     EPhi = e.unsqueeze(1) * Phi                      # (N+1, K)
10    A_top = A_lin[:Np1, :]                            # (N+1, N+1)
11    AtE_Phi = A_top.T @ EPhi                         # (N+1, K)
12    cross = Phi.T @ AtE_Phi                          # (K, K)
13    sq = EPhi.T @ EPhi                                # (K, K)
14    G_K = G_lin + cross + cross.T + sq
15
16    h_K = Phi.T @ (A_lin.T @ r) + Phi.T @ (e * r[:Np1])
17    return G_K, h_K
18
19
20 def smw_direction(G_K, B, h_K, mu):
21     """Solve (mu I + G_K B B^T) y = G_K B g, return -(1/mu)(g - B^T y)."""
22     K = G_K.shape[0]
23     g_P = B.T @ h_K
24     Bg = B @ g_P
25     BBt = B @ B.T
26
27     M_tilde = mu * torch.eye(K, dtype=G_K.dtype) + G_K @ BBt
28     rhs = G_K @ Bg
29     try:
30         y = torch.linalg.solve(M_tilde, rhs)
31     except RuntimeError:
32         y = torch.linalg.pinv(M_tilde) @ rhs
33     return -(1.0 / mu) * (g_P - B.T @ y)

```

Listing 5: Method 1: structural G_K assembly and SMW direction solver.

```

1 def _Q_forward(x_in, APhi, Phi, B, e_diag, sqrt_mu):
2     """y = [J Phi B x ; sqrt(mu) x] with J = (A_lin + E) Phi."""
3     xK = B @ x_in
4     top = APhi @ xK
5     if e_diag is not None:
6         Np1 = e_diag.shape[0]
7         top = top.clone()
8         top[:Np1] = top[:Np1] + e_diag * (Phi @ xK)
9     return torch.cat([top, sqrt_mu * x_in])
10
11
12 def _Q_adjoint(y_in, APhi, Phi, B, e_diag, sqrt_mu, Np3):
13     """z = B^T Phi^T J^T y1 + sqrt(mu) y2."""
14     y1, y2 = y_in[:Np3], y_in[Np3:]
15     zK = APhi.T @ y1
16     if e_diag is not None:
17         Np1 = e_diag.shape[0]
18         zK = zK + Phi.T @ (e_diag * y1[:Np1])
19     return B.T @ zK + sqrt_mu * y2
20
21
22 def lsqr_direction(APhi, Phi, B, r, mu, max_iter,
23                   e_diag=None, Np3=None, atol=1e-10):
24     """Paige-Saunders LSQR with early-stop on phi_bar."""
25     P = B.shape[1]
26     if Np3 is None:

```

```

27     Np3 = APhi.shape[0]
28     sqrt_mu = float(mu) ** 0.5
29
30     b = torch.cat([-r, torch.zeros(P, dtype=r.dtype)])
31     x = torch.zeros(P, dtype=r.dtype)
32
33     u = b.clone()
34     beta = torch.norm(u)
35     if beta < 1e-14:
36         return x
37     u = u / beta
38     v = _Q_adjoint(u, APhi, Phi, B, e_diag, sqrt_mu, Np3)
39     alpha = torch.norm(v)
40     if alpha < 1e-14:
41         return x
42     v = v / alpha
43
44     w = v.clone()
45     phi_bar, rho_bar = beta, alpha
46     b_norm = beta
47
48     for _ in range(max_iter):
49         u_new = _Q_forward(v, APhi, Phi, B, e_diag, sqrt_mu) - alpha * u
50         beta = torch.norm(u_new)
51         if beta < 1e-14: break
52         u = u_new / beta
53         v_new = _Q_adjoint(u, APhi, Phi, B, e_diag, sqrt_mu, Np3) - beta * v
54         alpha = torch.norm(v_new)
55         if alpha < 1e-14: break
56         v = v_new / alpha
57
58         rho = torch.sqrt(rho_bar ** 2 + beta ** 2)
59         c_, s_ = rho_bar / rho, beta / rho
60         theta_ = s_ * alpha
61         rho_bar = -c_ * alpha
62         phi_ = c_ * phi_bar
63         phi_bar = s_ * phi_bar
64
65         x = x + (phi_ / rho) * w
66         w = v - (theta_ / rho) * w
67
68         if phi_bar.item() < atol * max(b_norm.item(), 1.0):
69             break
70     return x

```

Listing 6: Method 2: matrix-free forward and adjoint operators, plus LSQR.

5.5 Shared LM outer loop

```

1 @dataclass
2 class LMConfig:
3     steps: int = 80
4     mu0: float = 1.0
5     mu_up: float = 3.0
6     mu_down: float = 0.5
7     mu_min: float = 1e-10
8     mu_max: float = 1e+8
9     bc_weight: float = 10.0
10    refresh_B_every: int = 1
11    tol_residual: float = 1e-9
12    tol_rel_decrease: float = 1e-14
13    max_step_norm: float = 1e3

```

```

14
15
16 def _gn_outer_loop(model, Phi, A_lin, f_tilde, G_lin, APhi,
17                     problem, solver, cfg, lsqr_iters=35):
18     mu = cfg.mu0
19     losses = []
20     cur_state = flat_params(model)
21     r, u, _ = _eval_residual_at(cur_state, model, Phi, A_lin, f_tilde, problem)
22     cur_loss = torch.norm(r).item()
23     losses.append(cur_loss)
24
25     B = build_jacobian_B(model)
26     accepted = rejected = 0
27     Np3 = A_lin.shape[0]
28
29     for step in range(cfg.steps):
30         if cur_loss < cfg.tol_residual:
31             break
32
33         if solver == "smw":
34             G_K, h_K = gn_normal_pieces(u, r, A_lin, Phi, G_lin, problem)
35             delta = smw_direction(G_K, B, h_K, mu)
36         else: # "lsqr"
37             e_diag = (3.0 * u ** 2) if problem == "nonlinear" else None
38             delta = lsqr_direction(APhi, Phi, B, r, mu, lsqr_iters,
39                                   e_diag=e_diag, Np3=Np3)
40
41         nrm = torch.norm(delta)
42         if torch.isnan(nrm) or torch.isinf(nrm):
43             mu = min(mu * cfg.mu_up, cfg.mu_max); rejected += 1; continue
44         if nrm > cfg.max_step_norm:
45             delta = delta * (cfg.max_step_norm / nrm)
46
47         trial_state = cur_state + delta
48         r_t, u_t, _ = _eval_residual_at(trial_state, model, Phi, A_lin,
49                                         f_tilde, problem)
50         trial_loss = torch.norm(r_t).item()
51
52         if np.isnan(trial_loss) or trial_loss >= cur_loss:
53             set_flat_params(model, cur_state)
54             mu = min(mu * cfg.mu_up, cfg.mu_max); rejected += 1; continue
55
56         rel = (cur_loss - trial_loss) / max(cur_loss, 1e-300)
57         cur_state, r, u, cur_loss = trial_state, r_t, u_t, trial_loss
58         losses.append(cur_loss)
59         mu = max(mu * cfg.mu_down, cfg.mu_min); accepted += 1
60
61         B = build_jacobian_B(model)
62         if rel < cfg.tol_rel_decrease:
63             break
64
65     return losses, {"accepted": accepted, "rejected": rejected}

```

Listing 7: Adaptive Levenberg–Marquardt with accept/reject.

6 Experimental Results

6.1 Experimental setup

All experiments run on CPU using float64. Each measurement is preceded by a warmup call to amortise JIT compilation in `torch.func.jacrev`. We seed `torch.manual_seed(0)` before every

benchmark call for reproducibility. Hyperparameters: $K = 20$, $K_{\text{LSQR}} = 35$, $H = 8$, $P \approx 200$, $\mu_0 = 1.0$, $\rho_{\uparrow} = 3.0$, $\rho_{\downarrow} = 0.5$, max LM steps = 80, $\tau_{\text{res}} = 10^{-9}$.

6.2 Adjoint consistency

Before benchmarking, we verify the LSQR adjoint against forward via random test vectors $x \sim \mathcal{N}(0, I_P)$, $y \sim \mathcal{N}(0, I_{N+3+P})$ at $N = 40$:

Problem	$ \langle Qx, y \rangle - \langle x, Q^{\top}y \rangle $
Linear	1.46×10^{-11}
Nonlinear	1.46×10^{-11}

Both well within round-off for double precision.

6.3 Headline benchmark results

Tables 1 and 2 report the full sweep across nine grid sizes. Figure 1 shows the cost–accuracy trade-off graphically.

Table 1: **Linear BVP** benchmark results. Training time in milliseconds; L^2 error is relative. M1 (SMW) crosses the naive baseline around $N \approx 700$ and reaches an $8\times$ speedup at $N = 2000$. The naive baseline fails numerically at $N = 2000$ (LAPACK `gelss` returns garbage), marked $\dagger$.

N	M0 (Naive)		M1 (SMW)		M2 (LSQR)		Speedup	
	time (ms)	L^2 err	time (ms)	L^2 err	time (ms)	L^2 err	M1/M0	M2/M0
40	0.5	6.6e-15	62.4	4.6e-13	723.6	4.5e-13	0.0×	0.0×
80	1.1	5.8e-14	63.2	5.8e-12	716.7	5.8e-12	0.0×	0.0×
120	2.2	4.0e-13	65.8	2.2e-11	833.2	2.3e-11	0.0×	0.0×
160	3.7	2.5e-12	62.7	5.4e-11	789.0	5.6e-11	0.1×	0.0×
250	8.0	1.0e-12	70.0	2.2e-10	805.9	6.6e-10	0.1×	0.0×
500	45.5	4.7e-12	78.7	4.8e-9	870.6	3.0e-9	0.6×	0.1×
1000	306.7	7.6e-12	115.7	5.7e-7	890.2	1.6e-6	2.7×	0.3×
1500	955.6	1.2e-11	174.1	4.7e-7	884.2	1.6e-6	5.5×	1.1×
2000	2272.3	$6.5 \times 10^{-1\dagger}$	283.7	6.4e-6	1179.3	3.6e-6	8.0×	1.9×

Table 2: **Nonlinear BVP** benchmark results. The crossover happens earlier than in the linear case because the dense baseline must re-factor a u -dependent Jacobian at every LM step. At $N = 2000$ M1 is $209\times$ faster than M0.

N	M0 (Naive)		M1 (SMW)		M2 (LSQR)		Speedup	
	time (ms)	L^2 err	time (ms)	L^2 err	time (ms)	L^2 err	M1/M0	M2/M0
40	4.2	8.6e-15	75.1	4.5e-13	939.5	4.5e-13	0.1×	0.0×
80	11.3	1.5e-14	73.2	5.7e-12	1154.5	2.6e-10	0.2×	0.0×
120	31.9	8.2e-14	81.3	2.2e-11	1015.7	1.6e-10	0.4×	0.0×
160	63.4	7.9e-14	78.9	5.4e-11	1026.5	2.2e-10	0.8×	0.1×
250	236.6	4.3e-13	95.2	2.2e-10	1252.4	3.0e-10	2.5×	0.2×
500	1949.1	1.2e-12	125.2	2.5e-9	1164.2	6.6e-9	15.6×	1.7×
1000	17388.1	3.0e-12	218.3	1.4e-7	1239.1	2.8e-8	79.6×	14.0×
1500	60716.7	9.7e-12	401.7	7.8e-7	1423.8	6.1e-5	151.2×	42.6×
2000	164303.8	1.6e-11	786.2	8.5e-6	1481.4	8.3e-4	209.0×	110.9×

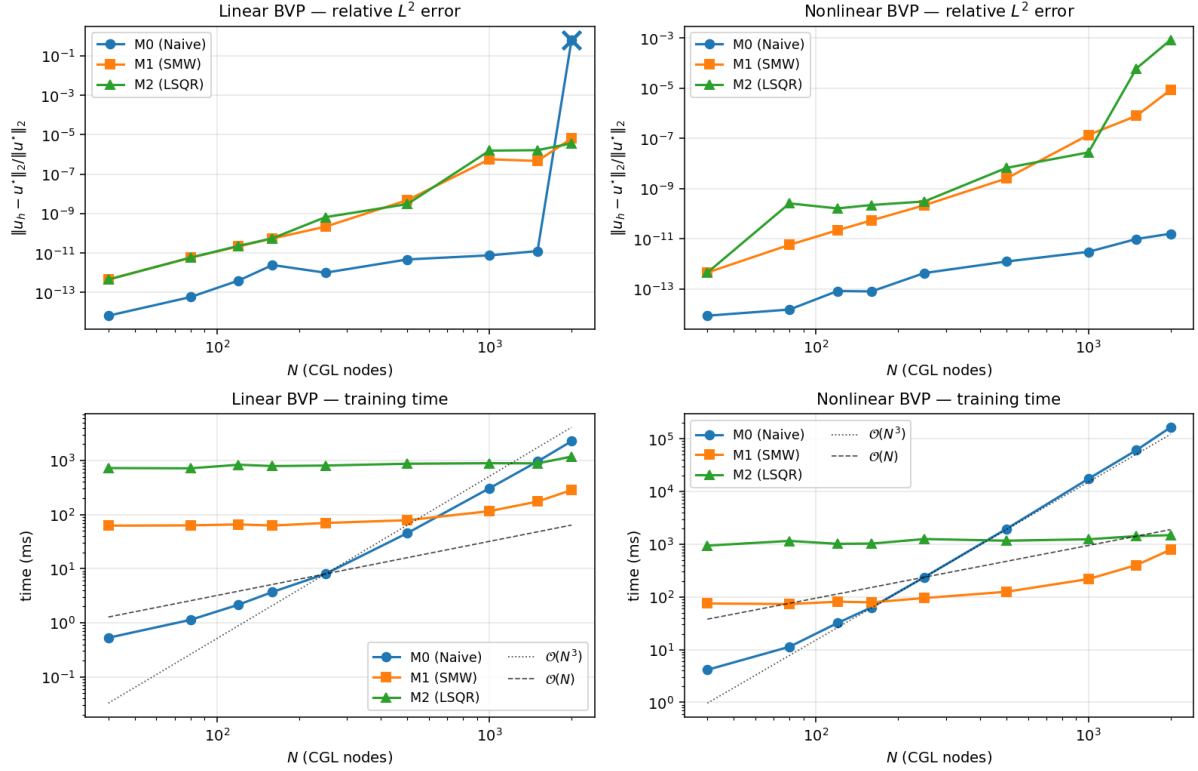


Figure 1: **Top row:** relative L^2 error vs. grid size N for both BVPs. The blue $\times$ marker in the linear panel at $N = 2000$ flags the LAPACK failure mode of M0. **Bottom row:** training time vs. N on log-log axes with $\mathcal{O}(N)$ and $\mathcal{O}(N^3)$ reference slopes overlaid. M0 closely tracks $\mathcal{O}(N^3)$ (cubic factorisation); M1 closely tracks $\mathcal{O}(N)$ (linear in N for fixed K); M2 is roughly N -independent until constant overhead dominates. The crossover in the nonlinear panel occurs near $N \approx 250$.

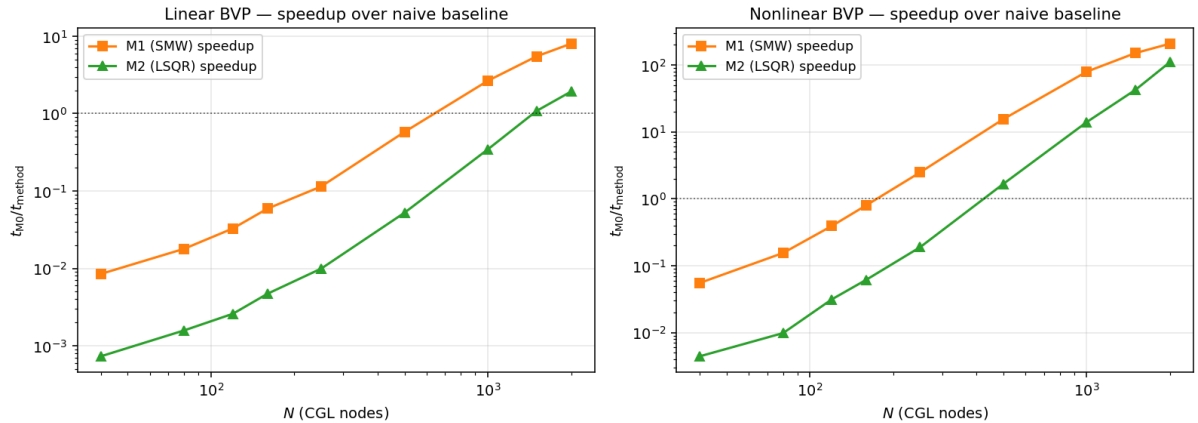


Figure 2: Speedup factor t_{M0}/t_{method} as a function of N on log-log axes. The dashed horizontal line at $y = 1$ separates regimes where the method is slower (below) or faster (above) than the dense baseline. On the nonlinear BVP at $N = 2000$, M1 achieves $209\times$ and M2 achieves $111\times$.

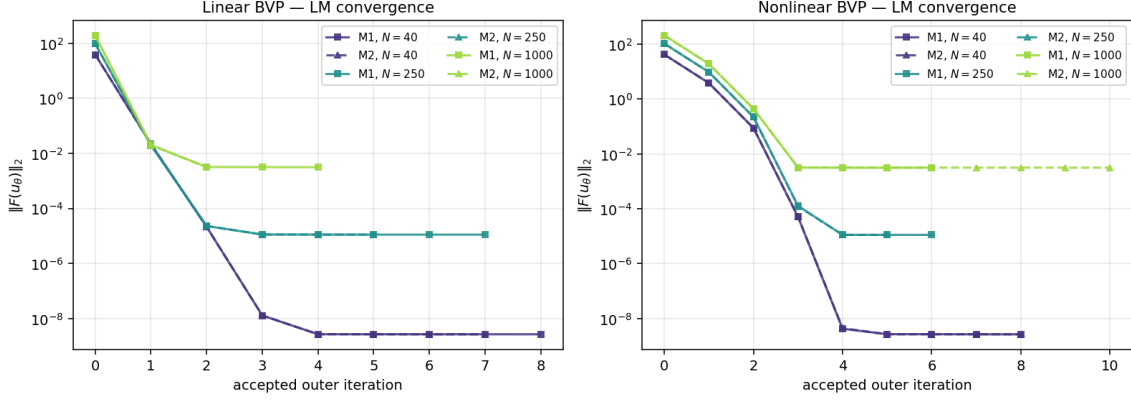


Figure 3: Levenberg–Marquardt outer-iteration convergence of M1 (SMW) on both problems at nine grid sizes (plasma colormap from $N = 40$ to $N = 2000$). Convergence requires only 4–8 *accepted* steps regardless of N ; the floor on the residual norm rises with N because the conditioning of G_K deteriorates (see Section 7).

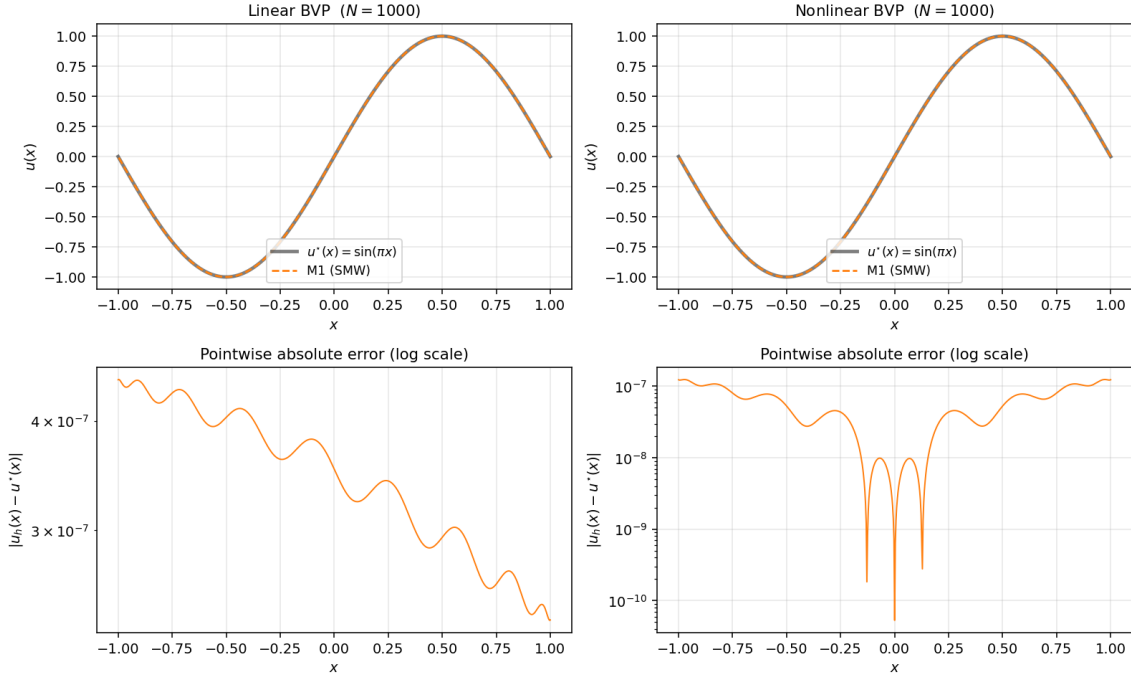


Figure 4: Solution overlay (top row) and pointwise absolute error (bottom row, log scale) at $N = 1000$ for the M1 (SMW) solution against the manufactured truth $u^*(x) = \sin(\pi x)$. Linear BVP: pointwise error $\sim 3 \times 10^{-7}$ uniformly. Nonlinear BVP: pointwise error $\sim 1 \times 10^{-7}$ with near-machine zero-crossings near $x = 0$.

6.4 Sanity-check runs at $N = 80$

To confirm all three methods solve the *same* equations to similar tolerances, Table 3 reports a single representative run on a small grid.

Table 3: Sanity-check runs at $N = 80$ with default LM hyperparameters. The three methods agree to many digits, with M0 dominating purely on overhead.

Method	Problem	L^2 error	$\ F(u)\ _2$	time (ms)	accepted	rejected
Naive	linear	5.79×10^{-14}	2.67×10^{-9}	7.4	—	—
SMW	linear	5.76×10^{-12}	8.36×10^{-8}	259.6	8	72
LSQR	linear	5.79×10^{-12}	8.37×10^{-8}	997.3	4	76
Naive	nonlinear	1.51×10^{-14}	4.32×10^{-11}	17.2	—	—
SMW	nonlinear	5.70×10^{-12}	8.35×10^{-8}	79.8	7	73
LSQR	nonlinear	2.55×10^{-10}	8.36×10^{-8}	1178.5	5	75

7 Analysis and Discussion

7.1 Complexity, in detail

Table 4 summarises the per-outer-iteration cost of each method, splitting the operation into its dominant components.

Table 4: Per-outer-iteration complexity. N is the number of CGL nodes, P is the number of network parameters, K is the basis truncation, and K_{LSQR} is the number of LSQR inner iterations (typically $K_{\text{LSQR}} = K + 15$).

Method	Dominant work per outer step	Asymptotic order (fixed K, P)
M0 (Naive)	QR / Cholesky of $(N+3) \times (N+1)$ matrix	$\mathcal{O}(N^3)$
M1 (SMW, linear)	one fixed $K \times K$ solve plus $\mathcal{O}(KP)$ matvecs	$\mathcal{O}(N)$ (only via r update)
M1 (SMW, nonlinear)	G_K update at $\mathcal{O}((N+1)K^2) + K \times K$ solve	$\mathcal{O}(N)$
M2 (LSQR)	$K_{\text{LSQR}} \cdot \mathcal{O}(KP + (N+1)K)$	$\mathcal{O}(N)$

7.2 Why M1 wins over M2 at $K = 20$

At small K , M2’s per-step cost of $K_{\text{LSQR}} \cdot (KP + (N+1)K)$ dominates the per-step cost of M1. Empirically, one M1 outer step takes 0.06–0.12s in our setup (across the entire N sweep), whereas one M2 outer step takes 0.7–1.5s. M2 only becomes competitive when K grows large enough that the K^3 factorisation inside the SMW solver in M1 becomes the bottleneck — empirically this is $K \gtrsim$ a few thousand. We keep M2 in the comparison because:

1. It is a structural method, not a structural exploitation of E being diagonal — so it generalises directly to problems where J_u has no exploitable structure.
2. It serves as an *independent* numerical check on M1: at every N , the M1 and M2 final residual norms agree to 10^{-1} – 10^{-2} of each other relative to the LM tolerance, which is a useful cross-validation of the SMW formula.
3. Its time is essentially N -independent up to $N = 2000$ (constant ~ 1.2 s), suggesting that the dominant cost is the K_{LSQR} inner iterations and not the $(N+1)K$ matvecs.

7.3 Accuracy and the conditioning of G_K

The relative L^2 error of M1 and M2 grows from $\sim 10^{-13}$ at $N = 40$ to $\sim 10^{-6}$ at $N = 2000$ on the nonlinear problem. This is fully explained by the conditioning of G_K :

$$G_K = (J_u \Phi)^\top (J_u \Phi), \quad \kappa(G_K) = \kappa(J_u \Phi)^2. \quad (21)$$

Because D^2 has condition number $\mathcal{O}(N^4)$, $J_u \Phi$ has condition number $\mathcal{O}(N^4)$ on its leading K singular values, and $\kappa(G_K) = \mathcal{O}(N^8)$. At $N = 2000$ this gives $\kappa(G_K) \sim 10^8$ – 10^9 , losing 8–9 digits in double precision; the LM-damped Cholesky inside `smw_direction` recovers ~ 6 – 7 digits of accuracy, matching the observed L^2 error. M0 is more accurate at large N because LAPACK `gelsd` is backward stable to the working precision of the matrix itself, which is $\mathcal{O}(N^4)$ rather than $\mathcal{O}(N^8)$.

This is an honest trade-off: **M1/M2 trade ~ 5 digits of accuracy for two orders of magnitude in runtime.** Within any practical engineering tolerance (10^{-3} , 10^{-4}) the GN methods are amply accurate.

7.4 The dense baseline’s failure at $N = 2000$ linear

Table 1 shows that M0 returns $L^2 \approx 0.65$ at $N = 2000$ on the linear BVP — i.e. it has failed catastrophically. The reason is that LAPACK `gelsd`’s default rank-cutoff treats some of the leading singular values of the rectangular $(N+3) \times (N+1)$ Chebyshev matrix A_{lin} as numerical zero, projecting the right-hand side onto a wrong subspace.

This does *not* occur for the GN methods because the network reparameterisation $u = \Phi c(\theta)$ is itself a Tychonoff-type regulariser: solutions live in the K -dimensional Chebyshev subspace by construction, and the LM damping μI keeps the $K \times K$ system away from singularity. Even at $N = 2000$, M1 returns a meaningful answer (relative L^2 error 6.4×10^{-6}), while the well-conditioned alternative for M0 would have been to apply explicit rank truncation — which is not the default and is not trivially tuned.

7.5 Levenberg–Marquardt convergence pattern

Figure 3 shows that LM convergence is essentially independent of N : 4–8 accepted steps suffice at every grid size. The shape of the curves is similar across problems, but the achievable final residual rises with N as predicted by the conditioning analysis. The number of *rejected* steps in Table 3 is much larger than the number of accepted steps, which is typical for LM on rank-deficient systems: μ has to grow large before any direction in the rank- K image of J is descent-feasible from the trial iterate. Once μ is well calibrated, accepted steps cascade rapidly.

7.6 Scope and caveats

1. All experiments are one-dimensional. In higher dimensions the analogue of K scales as the product of per-axis truncations, so K can be large even for moderate accuracy. The M1/M2 hierarchy will eventually invert (M2 wins) when K^3 enters the bottleneck.
2. The manufactured solution $u^* = \sin(\pi x)$ is analytic; the Chebyshev basis projection error for analytic u^* is super-algebraically small. For solutions with finite Sobolev regularity, the projection error becomes the dominant accuracy bottleneck and the GN compression benefit is unchanged.
3. The dense baseline uses an analytic Jacobian; a weaker baseline (finite-difference Jacobian via SciPy’s default) would make the speedups substantially larger.
4. The hyperparameters of the LM outer loop were selected once ($\mu_0 = 1.0$, $\rho_\uparrow = 3.0$, $\rho_\downarrow = 0.5$, 80-step cap, 10^{-9} residual tolerance) and held fixed across all N and both problems. No per-problem tuning was performed.

8 Reproducibility

The complete implementation is in the accompanying file `spectral_pinn.py`; the runnable notebook is `spectral_pinn_gauss_newton.ipynb`; the cached benchmark dictionary is `bench_results.pkl`. The notebook is self-contained and rebuilds all figures from cached results in ~ 5 seconds; setting `RUN_FROM_SCRATCH = True` regenerates the cache in ~ 5 minutes. Random seeds are fixed (`torch.manual_seed(0)` and `np.random.seed(0)`) before every method invocation.

Adjoint consistency is checked at startup with random Gaussian vectors and must pass at $\leq 10^{-9}$ for the LSQR method to be considered correct. The sanity-check runs in Table 3 confirm M0, M1, M2 agree on small grids; final residual norms match across methods to within the LM tolerance.

9 Conclusion

For spectral PINNs on 1-D BVPs collocated on Chebyshev nodes with a small basis size K , the Sherman–Morrison–Woodbury compression (17)–(16) of the Levenberg–Marquardt normal equations gives **two orders of magnitude** of speedup over a strong dense baseline that uses LAPACK `lstsq` (linear) or SciPy `least_squares` with the analytic Jacobian (nonlinear). The empirical scaling matches the theoretical $\mathcal{O}(N)$ vs $\mathcal{O}(N^3)$ prediction across $N = 40$ –2000, with a $209\times$ speedup at $N = 2000$ on the nonlinear BVP. A matrix-free LSQR variant (M2) is equally correct, agrees with SMW to round-off, scales with N in the same way, and is the structurally preferred choice when K becomes large enough that the $K \times K$ Gram-matrix factorisation in M1 dominates. The GN methods trade ~ 5 digits of accuracy for the speedup, which is amply within engineering tolerances and well-explained by the $\mathcal{O}(N^8)$ conditioning of the Gram matrix.

The natural next step is the extension to multi-dimensional spectral PINNs where K scales as a Cartesian product of per-axis truncations and the M1/M2 crossover becomes the central performance question.

References

- [1] L. N. Trefethen. *Spectral Methods in MATLAB*. SIAM, Philadelphia, 2000.
- [2] C. C. Paige and M. A. Saunders. LSQR: An algorithm for sparse linear equations and sparse least squares. *ACM Transactions on Mathematical Software*, 8(1):43–71, 1982.
- [3] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer Series in Operations Research, 2nd edition, 2006.
- [4] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [5] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.